How SaltStack compares with Puppet

	Salt	Puppet
File extensions	.sls	.pp
Base language	Jinja2-based YAML	Ruby-based DSL
Root structure	file roots (has options of Dev, Stage or Prod)	module path
Entry point	top.sls or init.sls	site.pp or init.pp
Client	Minion	Client
Augeaus module for file modifications	Not present	Present
Dashboard	No visual interface	Available in enterprise edition
Templating structure	Not that strong	Powerful
Dynamic Global Variables Declaration	Orderly but reduces flexibility; resides in separate location and cannot be declared on the fly. Variables are called pillars	More intuitive and flexible
Orchestration	ZeroMQ	Mcollective with no default functionality
Remote execution	Out-of-the-box features, supports dynamic querying and scalable orchestration	There is no option, as of now
Iterations	Loops within code are very flexible	Not that flexible; in fact, it is very tough to code loops for re-use of variables or call more than one value at the same time

set of state declarations.

Job: Tasks to be performed by Salt command execution.

Pillar: A simple key-value store for user-defined data to be made available to a minion. Often used to store and distribute sensitive data to minions.

Grain: A key-value pair which contains a fact about a system, such as its hostname or network addresses.

Jinja: A templating language framework for Python, which allows variables and simple logic to be dynamically inserted into static text files when they are rendered. Inspired by Django's templating language.

Salt key: Salt master manages which machines are allowed or not allowed to communicate with it.

Salt SSH: A configuration management and remote orchestration system, which does not require that any software besides SSH be installed on systems to be controlled.

SLS module: Contains a set of state declarations.

State declaration: A data structure that contains a unique ID and describes one or more states of a system, such as ensuring that a package is installed or a user is defined.

State module: A module that contains a set of state functions.

State run: The application of a set of states on a set of systems.

Target: Minion(s) to which a given Salt command will apply.

Setup

We can install SaltStack by three methods—Bootstrap, Yum or APT. We are only covering installs for the most common server platforms like Red Hat, SUSE and Debian, though the SaltStack official documentation includes Solaris, ArchLinux, Ubuntu and even Windows.

Bootstrap: This is the easiest way and it takes care of dependency packages as well on any platform.

Download the required script:

```
wget --no-check-certificate -O install_salt.sh http://
bootstrap.saltstack.org
```

Install the master:

```
sh install_salt.sh -M -N
```

Install the client:

```
sh install_salt.sh -A [Master's IP or DNS Record]
```

Yum: Yum is platform-independent and is best when we use Red Hat or SUSE. It will install all the required dependencies.

Install the master:

```
yum install salt-master
```

Now, install the client:

```
yum install salt-minion
```

For SUSE, replacing Yum with Zypper should also work.

```
zypper addrepo http://download.opensuse.org/repositories/
devel:languages:python/SLE_11_SP3/devel:languages:zypper
refresh
zypper install salt salt-minion salt-master
```